

Prim's Algorithm - MST

Node 중심 $\leftrightarrow$ edge 중심의 kruskal

행위:

현재 MST에 연결된 Node들에서 바깥으로
나가는 edge들에서 가장 작은 edge를 선택

$\Rightarrow$ MST

$\rightarrow$ MST에서 나가는 edge들

$\rightarrow$ 가장 작은 edge 선택

$\rightarrow$ 새 노드 추가

$\rightarrow$ 반복

"임의의 노드에서 시작해서 MST를 점점 확장한다.

매 단계마다 현재 MST안에 있는 Node와

MST 밖에 있는 Node를 연결하는 edge의

weight가 가장 작은 것을 선택

새로운 Node를 MST에 추가하면 모든 Node가
크기씩 늘어나 반복"

LeetCode: Min Cost to Connect Points
Prim's Version - 1 (min heap)

```
n = len(points)
adj = defaultdict(list)

for i in range(n):
    xi, yi = points[i]
    for j in range(i+1, n):
        xj, yj = points[j]
        man_dist = abs(xi - xj) + abs(yi - yj)
        adj[i].append((man_dist, j))
        adj[j].append((man_dist, i))

res = 0
visit = set()
min_heap = [(0, 0)]

while len(visit) < n:
    cost, node = heapq.heappop(min_heap)
    if node in visit:
        continue

    res += cost
    visit.add(node)
    for n_cost, neighbor in adj[node]:
        if neighbor not in visit:
            heapq.heappush((n_cost, neighbor))

return res
```

Prim's Version 2 - Array

$n = \text{len}(\text{points})$

$\text{node} = 0$

$\text{dist} = [10000000] * n$

$\text{visited} = [\text{False}] * n$

$\text{edges} = 0$

$\text{res} = 0$

while $\text{edges} < n - 1$:

$\text{visited}[\text{node}] = \text{True}$

$\text{nextNode} = -1$

 for i in $\text{range}(n)$:

 if $\text{visited}[i]$:
 continue

$\text{curr_dist} = (\text{abs}(\text{points}[i][0] - \text{points}[\text{node}][0]) + \text{abs}(\text{points}[i][1] - \text{points}[\text{node}][1]))$

$\text{dist}[i] = \min(\text{dist}, \text{curr_dist})$

 if $\text{nextNode} == -1$ or $\text{dist}[\text{nextNode}] > \text{dist}[i]$:

$\text{nextNode} = i$

$\text{res} += \text{dist}[\text{nextNode}]$

$\text{node} = \text{nextNode}$

$\text{edges} += 1$

return res

Prim's Algo

- almost same as Dijkstra

visited = [False] * n

min_heap = [(0, 0)] # cost, node

total = 0

while min_heap:

cost, node = heapq.heappop(—)

if visited[node]:
continue

total_cost += cost

for next_node, cost in graph[node]:

if not visited[next_node]:

heapq.heappush(—, (cost, next_node))

return total